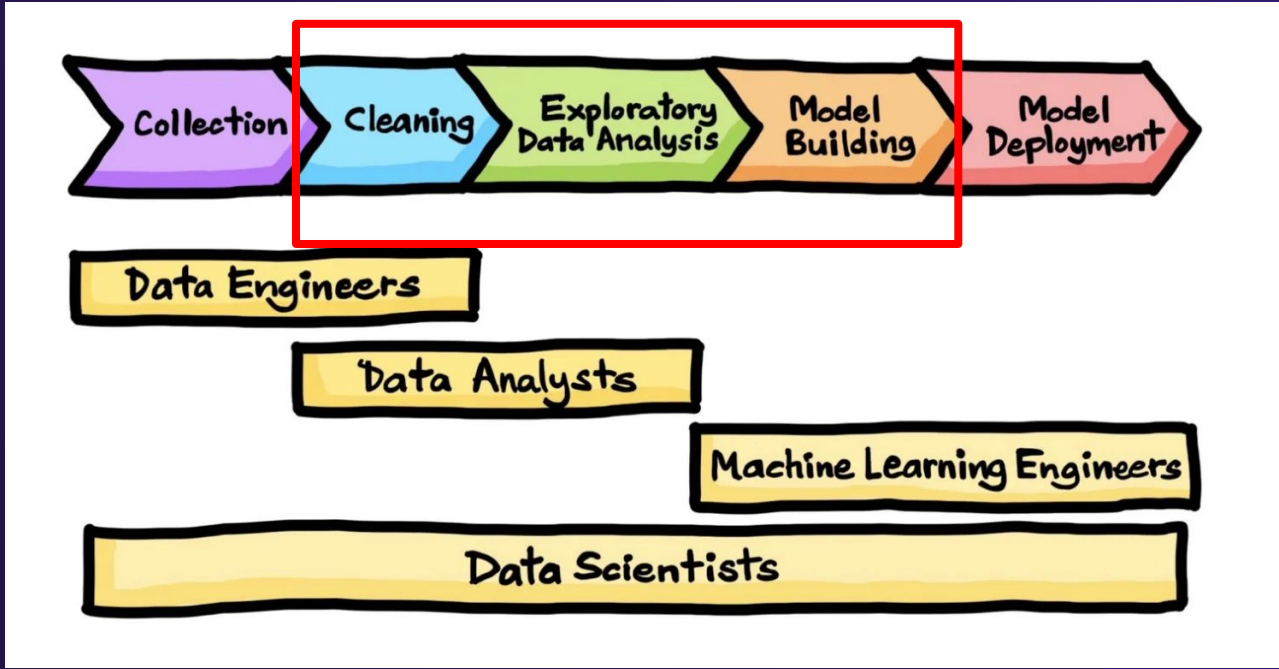
# LAB #1: DATA CLEANING

SUNDAY, 9/20

NDL
NITTANY DATA LABS

# ATTENDANCE - “DATA CLEANING”

# DATA SCIENCE PIPELINE



# WHY IS DATA CLEANING IMPORTANT?

- Converts raw data into organized, model-ready data
- Prevents errors and inconsistencies
- Lessens the chance of model bias
- Improves visibility of data during EDA

# WHAT IS "DIRTY DATA"

- Contains gaps, nulls, or duplicates
- Multiple data types in an attribute
- Few outliers that heavily bias summary statistics
- Any data that will cause errors during modeling

| county | year | county_ra | v001_raw | v001_num | v001_den | v001_cilo | v001_cihig | v001_race | v001_race |
|---|---|---|---|---|---|---|---|---|---|
| United States | 2019 | | 6900.63 | 3745538 | 9.08E+08 | 6890.349 | 6910.912 | | |
| Alabama | 2019 | | 9917.233 | 80440 | 13636816 | 9815.191 | 10019.27 | | |
| Autauga County | 2019 | 1 | 8824.057 | 815 | 156132 | 7935.326 | 9712.788 | 10471.25 | |
| Baldwin County | 2019 | 1 | 7224.632 | 2827 | 576496 | 6794.128 | 7655.136 | 10042.47 | 3086.606 |
| Barbour County | 2019 | 1 | 9586.165 | 451 | 72222 | 8200.118 | 10972.21 | 11332.56 | |
| Bibb County | 2019 | 1 | 11783.54 | 445 | 63653 | 10159.98 | 13407.11 | 14812.54 | |
| Blount County | 2019 | 1 | 10908.1 | 1050 | 161107 | 9895.583 | 11920.62 | | 5619.645 |
| Bullock County | 2019 | 1 | 12066.91 | 205 | 29266 | 9519.704 | 14614.12 | 14336.54 | |
| Butler County | 2019 | 1 | 14018.61 | 393 | 54804 | 11976.43 | 16060.78 | 15793.58 | |
| Calhoun County | 2019 | 1 | 12217.76 | 2333 | 321406 | 11484.59 | 12950.93 | 12955.02 | |
| Chambers County | 2019 | 1 | 11273.17 | 691 | 93770 | 9948.759 | 12597.58 | 10774.82 | |
| Cherokee County | 2019 | 1 | 11294.41 | 575 | 71014 | 9767.023 | 12821.8 | 16395.25 | |
| Chilton County | 2019 | 1 | 10831.92 | 850 | 123617 | 9733.842 | 11929.99 | 12000.44 | |
| Choctaw County | 2019 | 1 | 12047.19 | 273 | 35489 | 9740.13 | 14354.24 | 12215.47 | |
| Clarke County | 2019 | 1 | 9688.374 | 412 | 67037 | 8207.626 | 11169.12 | 12447.32 | |
| Clay County | 2019 | 1 | 9666.041 | 271 | 36916 | 7821.948 | 11510.13 | 12628.16 | |
| Cleburne County | 2019 | 1 | 11985.56 | 334 | 41389 | 9955.606 | 14015.5 | | |
| Coffee County | 2019 | 1 | 7933.13 | 743 | 143899 | 7085.258 | 8781.002 | 10333.51 | 6033.742 |
| Colbert County | 2019 | 1 | 10332.48 | 1016 | 149595 | 9321.505 | 11343.46 | 11604.94 | |
| Conecuh County | 2019 | 1 | 10635.37 | 272 | 34170 | 8546.834 | 12723.9 | 11100.61 | |

# UNDERSTANDING THE DATA

- Difficult to make corrections without understanding data structure
- Look at column names, data types, summary statistics

Summary Statistics

| Variable | Obs. | Mean | Std. Dev. | Min | Max |
|---|---|---|---|---|---|
| English | 1670 | .588 | .492 | 0 | 1 |
| Male | 1670 | 0.741 | .438 | 0 | 1 |
| Age | 1670 | 2.169 (25-35) | .833 | 1 | 4 |
| Final Grades | 1410 | 11.649 | 2.858 | 3.288 | 19.971 |
| Complete any assign. | 23918 | .138 | .345 | 0 | 1 |
| Completion | 23918 | .0602 | .458 | 0 | 1 |

# NULL/DUPLICATE VALUES

- Can either drop nulls or replace with mean/median
- Depends on how many nulls and how much variance matters
- Search for duplicates and remove

```
In [7]: df_students.isnull().sum()

Out[7]: First        0
        Last         0
        Math         2
        Physics      3
        Computers    4
        dtype: int64
```

```
In [155]: #drop the records with age missing in inp0 and copy in inp1 dataframe.
          inp0['age'].dropna(inplace=True)

In [156]: inp0['age'].isnull().sum()

Out[156]: 0

In [157]: inp0.isnull().sum()

Out[157]: age          20
          salary        0
          balance       0
          marital       0
          targeted      0
          default       0
```

# FORMATTING STRINGS

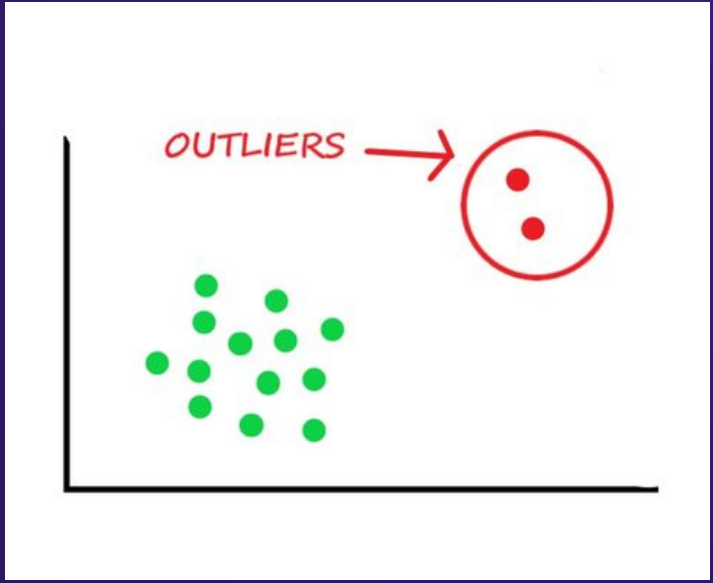
- Raw data often has string matching issues
  - ex. “New York” vs. “new york” vs. “NY”
- Use functions to homogenize strings

```
In [1]: a = 'MuG'
        a.lower()

Out[1]: 'mug'
```

# OUTLIERS

- Outliers can bias the mean which could negatively affect model output
- Criteria:
  - < Q1 - (1.5 * IQR)
  - > Q3 - (1.5 * IQR)



# FIXING STRUCTURE

- Sometimes the data is structured in an unusable way
- Splitting columns may be needed
  - ex. extracting month from date

```python
current_date = date.today()
print("Year:", current_date.year)
print("Month:", current_date.month)
print("Day:", current_date.day)
```

```
Year: 2023
Month: 2
Day: 18
```

Left Table

| A | B | C |
|---|---|---|
| A2 | B2 | C2 |
| A3 | B3 | C3 |
| A4 | B4 | C4 |

Right Table

| C | D |
|---|---|
| C1 | D1 |
| C2 | D2 |
| C4 | D4 |
| C5 | D5 |

Result Table

| A | B | C | D |
|---|---|---|---|
| | | C1 | D1 |
| A2 | B2 | C2 | D2 |
| A3 | B3 | C3 | |
| A4 | B4 | C4 | D4 |
| | | C5 | D5 |

# WORDS OF ADVICE

- Make edits carefully, and document them
- Never alter the raw data itself; create a copy or new data frame
- Understanding the data and what you want to model is crucial to successful cleaning

# COLAB

# CODE IT YOURSELF!

Click this link to download our Retail Stores Dataset.

```
sales_df_cleaned.shape
```

```
(11362, 11)
```

```
# Show null counts
null_counts = sales_df_cleaned.isnull().sum()
print(null_counts)
```

```
Transaction ID      0
Customer ID         0
Category            0
Item                0
Price Per Unit      0
Quantity            0
Total Spent         0
Payment Method      0
Location            0
Transaction Date    0
Discount Applied    0
dtype: int64
```

```
# Show data types of each column
sales_df_cleaned.dtypes
```

| | 0 |
|---|---|
| Transaction ID | object |
| Customer ID | object |
| Category | object |
| Item | object |
| Price Per Unit | float64 |
| Quantity | float64 |
| Total Spent | float64 |
| Payment Method | object |
| Location | object |
| Transaction Date | datetime64[ns] |
| Discount Applied | object |

```
# Show summary statistics for the numerical data
sales_df_cleaned.describe(include="number")
```

| | Price Per Unit | Quantity | Total Spent |
|---|---|---|---|
| count | 11362.000000 | 11362.000000 | 11362.000000 |
| mean | 23.356847 | 5.535029 | 129.642537 |
| std | 10.736030 | 2.855166 | 94.662368 |
| min | 5.000000 | 1.000000 | 5.000000 |
| 25% | 14.000000 | 3.000000 | 52.000000 |
| 50% | 23.000000 | 6.000000 | 108.500000 |
| 75% | 33.500000 | 8.000000 | 192.000000 |
| max | 41.000000 | 10.000000 | 410.000000 |

# CODE IT YOURSELF!

```
# Impute 'Unknown' for missing discount
sales_df['Discount Applied'] = sales_df['Discount Applied'].fillna('Unknown')
```

```
# Drop rows with 2 or more nulls
sales_df_cleaned = sales_df.dropna(thresh=10)
sales_df_cleaned.shape
```

```
(11362, 11)
```

# THANKS FOR COMING!

NEXT MEETING:
9/27 - EXPLORATORY DATA ANALYSIS